

| Rol: Semi-Senior FullStack Python Developer

Candidato: Franco **Fecha de inicio:** Sábado 21 de marzo de 2026 **Fecha de entrega:** Lunes 23 de marzo de 2026 **Repositorio:**
<https://github.com/francoasevey/PythonCodeChallengeGraphQL-NLP>

1. Comprensión del Challenge

El challenge requiere construir una API funcional, dockerizada y documentada que integre tres servicios:

- **Data Service:** expone un endpoint GraphQL sobre un dataset CSV y un endpoint NLP que acepta preguntas en lenguaje natural y responde en lenguaje natural.
- **Docs Service:** provee documentación unificada Swagger/OpenAPI 3.0 con ejemplos reales del dataset.
- **Auth Service** (*opcional, se implementa*): OAuth2 client credentials flow que emite JWT requerido como Bearer token en todos los endpoints del Data Service.

El criterio de evaluación implícito de un challenge a nivel Semi-Senior/Senior incluye: claridad de arquitectura, calidad de commits, documentación, y decisiones técnicas justificadas — no solo que el código funcione.

2. Decisiones de Arquitectura

2.1 Distribución del sistema

Se opta por una arquitectura de **microservicios con 3 contenedores Docker separados**. Esta decisión está alineada con el enunciado, que define servicios con responsabilidades distintas (datos, documentación, autenticación), y permite escalar o reemplazar cada servicio de forma independiente.

2.2 Organización interna

Se aplica una **N-Layer simplificada** (router → service → data) dentro de cada servicio. Se descarta Clean Architecture y Hexagonal deliberadamente: para el scope de este challenge agregan burocracia sin valor observable. El objetivo es código legible y estructurado, no sobre-ingenierizado.

2.3 Patrones aplicados

Patrón	Aplicación concreta
SOLID	Cada archivo tiene una única responsabilidad; un archivo con imports de más de 2 módulos distintos es señal de mezcla
Repository pattern	csv_service.py actúa como única fuente de verdad del CSV
Dependency Injection	FastAPI Depends() para inyección de servicios y validación de token
DTO pattern	Pydantic v2 modela todas las entradas y salidas de los endpoints

2.4 Decisiones técnicas clave

docs_service — Agregación dinámica de specs (Opción A) docs_service no mantiene una spec estática. En su lugar, hace GET /openapi.json a data_service y auth_service en startup, fusiona los paths y schemas, y sirve un Swagger UI unificado. Esto refleja un patrón real de API Gateway de documentación.

NLP — Contexto inteligente con pandas, no raw CSV El CSV no se manda completo a Claude (riesgo de exceder el contexto y hallucinations). En su lugar, csv_service.py expone una función build_nlp_context() que construye un resumen estructurado con pandas: cantidad de filas, columnas, muestra de 5 filas, y estadísticas clave (rango de fechas, clientes únicos, top categorías, top marcas). Este string va como system prompt a Claude; la pregunta del usuario va como user message.

GraphQL + JWT — Validación via context_getter de Strawberry Strawberry maneja su propio request lifecycle. El middleware HTTP estándar de FastAPI no intercepta correctamente el endpoint /graphql. La validación del token se realiza mediante context_getter, que recibe el request con Depends verify_token) y pasa el usuario validado al contexto del resolver. Esto también permite que el playground de GraphQL funcione sin conflictos.

Docker — depends_on con healthcheck Cada servicio expone un endpoint /health. El docker-compose.yml usa condition: service_healthy para garantizar el orden de startup correcto: auth_service levanta primero, data_service espera que esté healthy, docs_service espera a ambos. Evita race conditions en la demo.

3. Stack Tecnológico

Componente	Tecnología	Justificación
------------	------------	---------------

Framework API	FastAPI	Performance, tipado nativo, OpenAPI automático
GraphQL	Strawberry	Type hints nativos de Python, integración directa con FastAPI
NLP	Claude API (Anthropic)	Respuestas reales en lenguaje natural; diferenciador frente a soluciones de filtros por keywords
JWT / OAuth2	python-jose	Standard, liviano, compatible con FastAPI's OAuth2 utilities
Validación y DTOs	Pydantic v2	Más rápido que v1, mejor integración con FastAPI moderno
Procesamiento CSV	pandas	Lectura eficiente + agregaciones para contexto NLP
Containerización	Docker + docker-compose	Requerimiento del challenge
Configuración	python-dotenv + .env	Sin secrets hardcodeados en código

4. Estructura de Carpetas

```

challenge/
├── auth_service/
│   ├── main.py           # App FastAPI + rutas /token y /health
│   ├── jwt_handler.py    # Generación y validación de JWT
│   ├── schemas.py        # DTOs: TokenRequest, TokenResponse
│   └── config.py          # Variables de entorno del servicio
├── data_service/
│   ├── main.py           # App FastAPI + montaje de routers
│   ├── routers/
│   │   ├── graphql.py    # Endpoint /graphql con context_getter para JWT
│   │   └── nlp.py         # Endpoint /nlp con validación de token
│   └── services/
│       └── csv_service.py # Carga CSV, expone datos crudos y
build_nlp_context()
├── nlp_service.py        # Cliente Claude API
├── schema/
├── types.py              # Strawberry types con columnas mapeadas a
nombres legibles
├── models/
└── schemas.py            # Pydantic DTOs + ErrorResponse estándar

```

```

├── middleware/
│   ├── auth.py          # verify_token() para uso con Depends()
│   └── config.py        # Variables de entorno del servicio
├── docs_service/
│   ├── main.py          # App FastAPI + Swagger UI unificado
│   └── aggregator.py    # Fetch y merge de /openapi.json de cada servicio
├── data/
│   └── dataset.csv
├── docker-compose.yml   # 3 servicios con healthcheck y depends_on
├── .env.example         # Todas las variables documentadas
└── README.md           # Setup, variables, ejemplos curl, arquitectura

```

5. Endpoints a Implementar

Auth Service — Puerto 8002

Método	Endpoint	Descripción
POST	/token	OAuth2 client credentials — retorna JWT
GET	/health	Health check del servicio

Data Service — Puerto 8001

Método	Endpoint	Auth	Descripción
POST	/graphql	Bearer JWT	Queries GraphQL sobre el dataset CSV
GET	/graphql	No*	GraphQL playground (GraphiQL)
POST	/nlp	Bearer JWT	Consulta en lenguaje natural — responde con Claude API
GET	/health	No	Health check del servicio
GET	/stats	Bearer JWT	Estadísticas del dataset (bonus)

*GET /graphql es accesible sin token por diseño — el playground es una herramienta de desarrollo, no un endpoint de datos. Las queries ejecutadas desde él (POST /graphql) sí requieren Bearer token. Estándar de industria: Apollo Server, Hasura.

Queries GraphQL planificadas:

- `productInteractions(limit, offset)` — listado paginado
- `productsByCategory(category)` — filtro por categoría
- `interactionsByDevice(deviceType)` — *descartada: la columna `id_ga_tipo_dispositivo` contiene hashes numéricos no legibles, filtrar por ella no aporta valor*
- `topBrands(limit)` — marcas con más interacciones
- `interactionsByDateRange(from, to)` — rango de fechas

Docs Service — Puerto 8000

Método	Endpoint	Descripción
GET	/docs	Swagger UI con spec unificada
GET	/openapi.json	Spec fusionada de todos los servicios
GET	/health	Health check del servicio

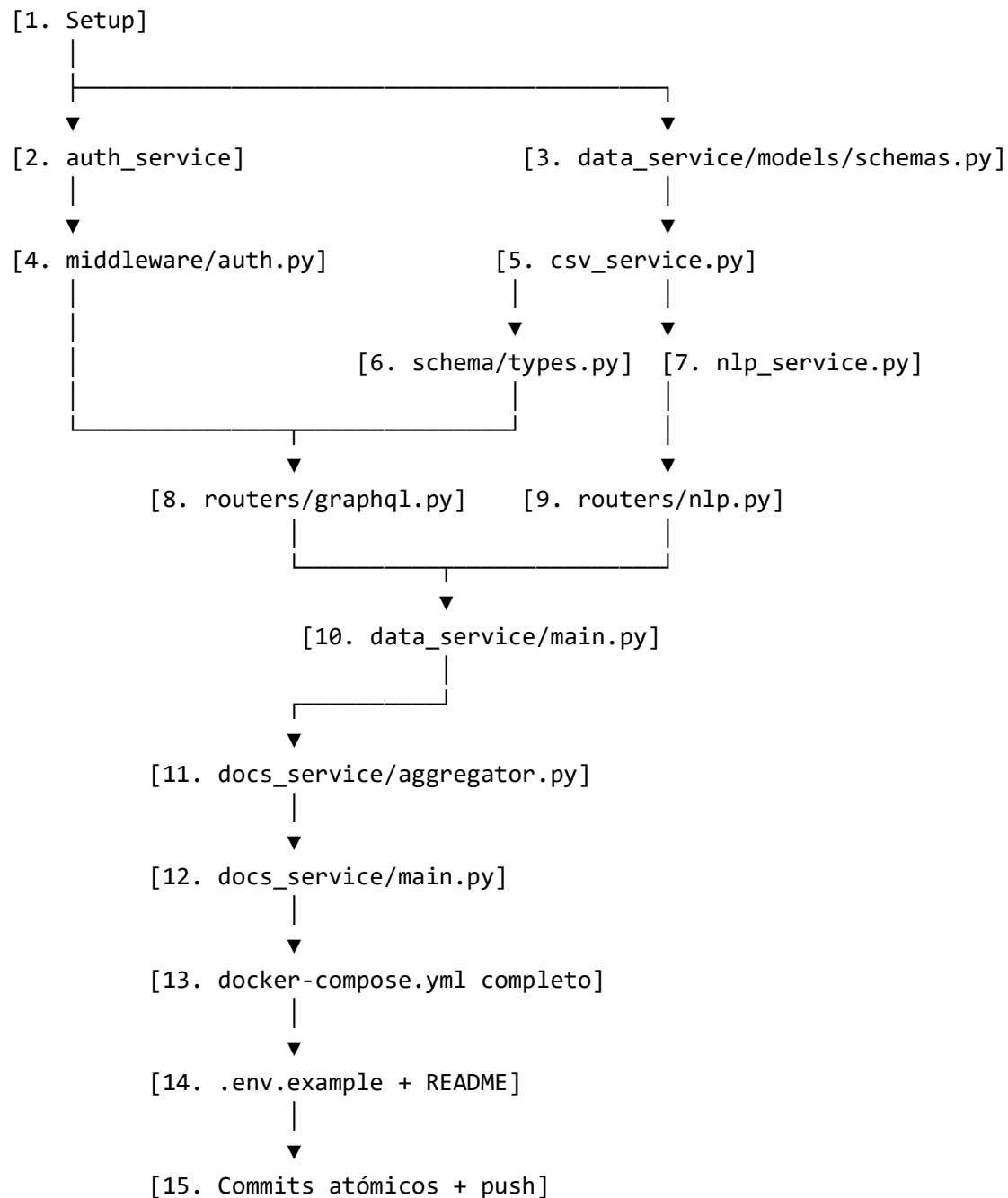
6. Estimación de Tiempos

Tarea	Estimación	Sesión
Setup inicial: repo, estructura de carpetas, docker-compose base	1h	Sábado AM
auth_service completo: JWT, OAuth2 flow, testeado con curl	2h	Sábado AM
csv_service: carga CSV + <code>build_nlp_context()</code>	1h	Sábado PM
GraphQL: schema Strawberry, types, queries con filtros	3h	Sábado PM
NLP endpoint: Claude API client + integración con csv_service	3h	Domingo AM
docs_service: aggregator dinámico + Swagger UI unificado	2h	Domingo PM
README nivel producción + <code>.env.example</code> + revisión final	1h	Domingo N

7. Flujo de Trabajo — Orden de Tareas

El orden de implementación está definido por dependencias reales entre archivos y servicios. Una tarea no puede comenzar hasta que sus dependencias estén completas y testeadas.

7.1 Grafo de dependencias



7.2 Detalle de cada tarea

#	Tarea	Archivos	Depende de	Bloquea
1	Setup inicial	estructura de carpetas, Dockerfiles vacíos, docker-compose skeleton, .gitignore, repo GitHub	—	Todo
2	auth_service completo	config.py, schemas.py, jwt_handler.py, main.py	1	4
3	DTOs base de data_service	models/schemas.py (ErrorResponse, NLPRequest, NLPResponse)	1	8, 9
4	Middleware de autenticación	middleware/auth.py (verify_token con Depends)	2	8, 9
5	CSV service	services/csv_service.py (carga DataFrame + build_nlp_context())	1	6, 7
6	GraphQL types	schema/types.py (Strawberry types con columnas mapeadas)	5	8
7	NLP service	services/nlp_service.py (Claude API client)	5	9
8	GraphQL router	routers/graphql.py (schema + queries + context_getter JWT)	3, 4, 6	10
9	NLP router	routers/nlp.py (endpoint POST /nlp con Depends(verify_token))	3, 4, 7	10
10	data_service main	data_service/main.py (monta routers + /health)	8, 9	11
11	Docs aggregator	docs_service/aggregator.py (fetch + merge /openapi.json)	10	12
12	Docs main	docs_service/main.py (Swagger UI unificado + /health)	11	13
13	Docker compose final	docker-compose.yml (healthchecks + depends_on + volumes)	12	14
14	Cierre	.env.example, README.md	13	15
15	Entrega	commits atómicos revisados + push + repo público	14	—

7.3 Tareas que se pueden ejecutar en paralelo

Dado que el desarrollo es secuencial (un desarrollador), se identifican los bloques que no tienen dependencia entre sí y se pueden intercalar sin riesgo:

- **Tareas 2 y 3** no se bloquean entre sí — `auth_service` y los DTOs de `data_service` son totalmente independientes.
- **Tareas 6 y 7** no se bloquean entre sí — ambas dependen de `csv_service` (tarea 5) pero no entre ellas.
- **Tareas 8 y 9** no se bloquean entre sí — ambas dependen de `middleware` (tarea 4) y DTOs (tarea 3) pero no entre ellas.

7.4 Regla de avance

Una tarea se considera **completada** cuando su endpoint o función puede ser llamado y devuelve la respuesta esperada. No se avanza a la siguiente tarea hasta cumplir este criterio. Esto garantiza que las dependencias aguas abajo siempre parten de una base funcional.

8. Estrategia de Commits

Los commits serán atómicos, progresivos y reales — no un commit final de todo el proyecto. Se usarán prefijos convencionales:

```
feat: inicializa estructura del proyecto y docker-compose base
feat(auth): implementa OAuth2 client credentials con JWT
feat(data): agrega csv_service con carga de dataset y contexto NLP
feat(graphql): implementa schema Strawberry con tipos del dataset
feat(graphql): agrega queries de filtrado por categoría, dispositivo y fecha
feat(nlp): integra Claude API con contexto inteligente del CSV
feat(docs): implementa aggregator dinámico de specs OpenAPI
chore: agrega .env.example y README con ejemplos curl
```

9. Riesgos y Mitigaciones

Riesgo	Probabilidad	Mitigación
Endpoint NLP con requerimientos ambiguos	Alta	Decisión tomada: Claude API con contexto pandas resumido — no keywords
Commits no progresivos (presión de tiempo)	Media	Committear por feature terminado, no al final

Race condition entre contenedores al levantar	Media	depends_on con condition: service_healthy en docker-compose
Exceder contexto de Claude con CSV completo	Media	build_nlp_context() con resumen estructurado, no raw CSV
Strawberry + JWT middleware incompatible	Baja	JWT validado via context_getter, no middleware HTTP

10. Criterios de Calidad

- **Una responsabilidad por archivo** — máximo ~150 líneas, máximo ~30 líneas por función
- **Sin secrets en código** — todo por variables de entorno, .env en .gitignore
- **Error handling consistente** — ErrorResponse Pydantic estándar en todos los endpoints
- **/health en cada servicio** — usado también por docker-compose para healthchecks
- **Swagger con ejemplos reales del dataset** — no ejemplos genéricos
- **README ejecutable** — cualquier persona puede levantar el proyecto con un docker-compose up

11. Herramientas de IA Utilizadas

Claude (Anthropic) — claude-sonnet-4-6 Utilizado como asistente técnico durante la planificación y el desarrollo del challenge. Las conversaciones completas se adjuntan como PDF según los lineamientos del enunciado.

Usos concretos durante el desarrollo:

- Revisión y mejora del plan de arquitectura
- Identificación de riesgos técnicos (Strawberry + JWT, contexto NLP, docs aggregation)
- Generación asistida de código con revisión manual
- Como modelo de lenguaje detrás del endpoint /nlp en producción (Claude API)

Documento generado como parte del proceso de planificación previo al desarrollo. Refleja decisiones técnicas tomadas antes de escribir la primera línea de código.